

Exercise: Simple Tasks API with FastAPI + MongoDB

Build a **Tasks Management API** using FastAPI and MongoDB with a single tasks collection. Focus on basic CRUD and simple query filters.

Step 1: Project Setup (10 min)

Create the folder structure:

```
tasks_api/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── models.py      # Pydantic schemas
│   ├── database.py    # Mongo connection
│   ├── crud.py        # DB operations
│   └── routes.py      # API endpoints
├── requirements.txt   # fastapi, uvicorn, pymongo
└── README.md
```

requirements.txt:

```
fastapi
uvicorn
pymongo
python-dotenv
```

Commands:

- Create venv (optional), install deps: `pip install -r requirements.txt`.
- Run: `uvicorn app.main:app --reload`.

Step 2: Design Task Schema (10–15 min)

Model a **task** document like this:

```
{
  "_id": ObjectId("..."),
  "title": "Finish FastAPI homework",
  "description": "Implement basic CRUD for tasks",
  "status": "pending",          // "pending", "in_progress",
                                "done"
  "priority": 2,                // 1=low, 2=medium, 3=high
  "due_date": ISODate("2026-02-10T18:30:00Z")
}
```

In `models.py`, define:

- TaskCreate
 - title: str (min length 3)
 - description: str | None
 - priority: int (1–3)
 - due_date: datetime | None
- TaskUpdate
 - All fields optional: title, description, status, priority, due_date
- TaskResponse
 - Includes `_id`: str, plus all fields above.
- Add a validator so status is only "pending", "in_progress", or "done" (you can use `Literal` or `Enum`).

Step 3: Mongo Connection (5 min)

In `database.py`:

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017")
db = client.tasks_db
tasks_col = db.tasks
```

(Optional) Insert 3–5 sample tasks manually via Mongo shell/Compass or a quick script if time permits.

Step 4: CRUD Functions in crud.py (15–20 min)

Implement these functions:

- `create_task(payload: dict) -> dict`
 - Insert the document into `tasks_col`.
 - Return the inserted document with `_id` converted to `str`.
- `list_tasks(status: str | None = None, priority: int | None = None) -> list[dict]`
 - Build a filter dict dynamically.
 - Example filters:
 - `{"status": status}` if provided.
 - `{"priority": priority}` if provided.
 - Return a list of tasks, each with `_id` as `str`.
- `get_task(task_id: str) -> dict | None`
 - `find_one({"_id": ObjectId(task_id)})`.
 - If not found, return `None`.
- `update_task(task_id: str, payload: dict) -> dict | None`
 - Use `update_one({"_id": ObjectId(task_id)}, {"$set": payload})`.
 - Return updated document (you can use `find_one` again).
- `delete_task(task_id: str) -> bool`
 - Use `delete_one(...)`.
 - Return `True` if something was deleted, else `False`.

Keep error handling simple; you can return `None/False` and let the routes raise HTTP errors.

Step 5: Routes with FastAPI (15–20 min)

In `routes.py`:

```
from fastapi import APIRouter, HTTPException
from .models import TaskCreate, TaskUpdate, TaskResponse
from .crud import create_task, list_tasks, get_task, update_task,
    delete_task

router = APIRouter(prefix="/tasks", tags=["tasks"])

@router.post("/", response_model=TaskResponse)
async def create_task_route(payload: TaskCreate):
    task = create_task(payload.dict())
```

```

        return task

@router.get("/", response_model=list[TaskResponse])
async def list_tasks_route(status: str | None = None, priority:
    int | None = None):
    tasks = list_tasks(status=status, priority=priority)
    return tasks

@router.get("/{task_id}", response_model=TaskResponse)
async def get_task_route(task_id: str):
    task = get_task(task_id)
    if not task:
        raise HTTPException(status_code=404, detail="Task not
            found")
    return task

@router.put("/{task_id}", response_model=TaskResponse)
async def update_task_route(task_id: str, payload: TaskUpdate):
    updated = update_task(task_id, {k: v for k, v in
        payload.dict().items() if v is not None})
    if not updated:
        raise HTTPException(status_code=404, detail="Task not
            found")
    return updated

@router.delete("/{task_id}")
async def delete_task_route(task_id: str):
    ok = delete_task(task_id)
    if not ok:
        raise HTTPException(status_code=404, detail="Task not
            found")
    return {"deleted": True}

```

In main.py:

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from .routes import router as tasks_router

app = FastAPI(title="Tasks Mongo API")

app.add_middleware(
    CORSMiddleware,

```

```
        allow_origins=["*"],
        allow_methods=["*"],
        allow_headers=["*"],
    )

app.include_router(tasks_router)
```

Step 6: Minimal Test Checklist (5–10 min)

Using Swagger UI (<http://localhost:8000/docs>):

1. POST /tasks/ – create 1–2 tasks.
2. GET /tasks/ – see all tasks.
3. GET /tasks/?status=pending – filter by status.
4. PUT /tasks/{task_id} – change status to "done".
5. DELETE /tasks/{task_id} – delete a task and verify with GET /tasks/.